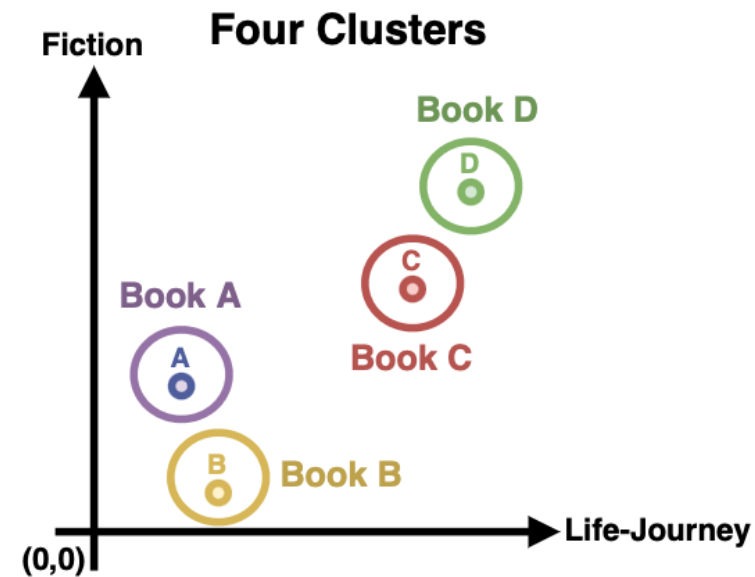
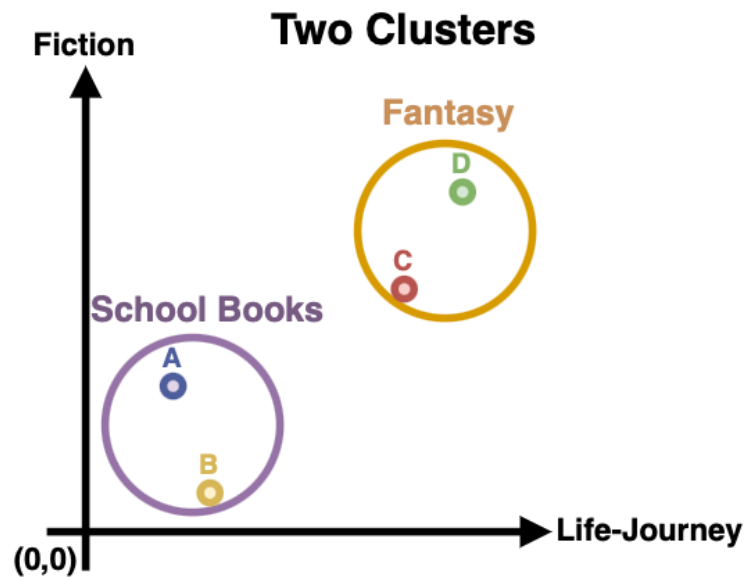
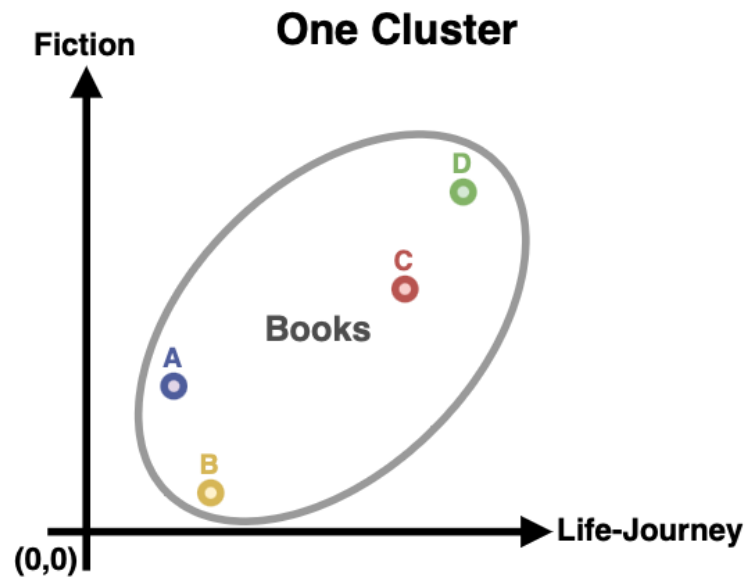
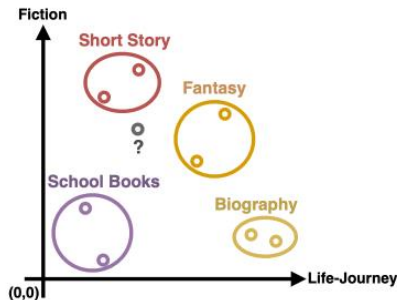




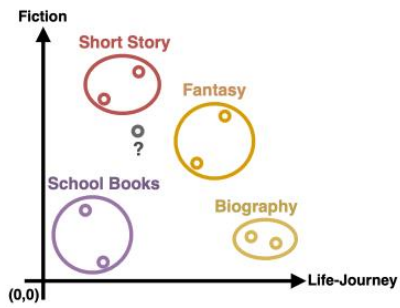
Clustering II



K-Means: Resumen

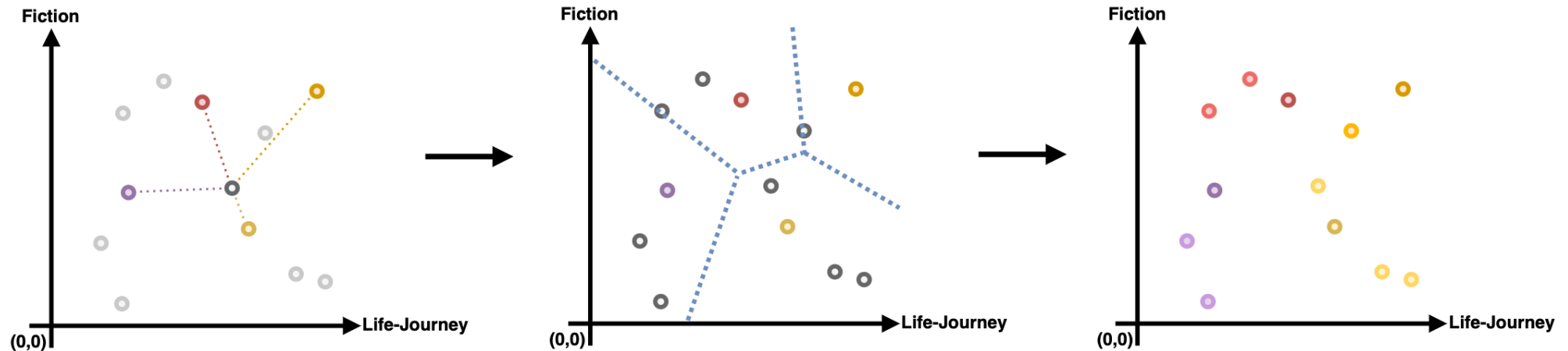
1. En el primer paso, los puntos se colocan aleatoriamente.
2. Se realiza la media de cada atributo y se calculan distancias.

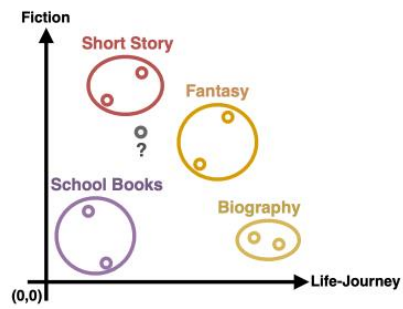
$$\text{mean}(\text{cluster}_j) = \frac{\sum_{\text{point}}^{\text{cluster}_j, \text{points}} \text{point}}{\text{len}(\text{cluster}_j, \text{points})}$$



K-Means: Resumen

1. En el primer paso, los puntos se colocan aleatoriamente.
2. Se realiza la media de cada atributo y se calculan distancias.



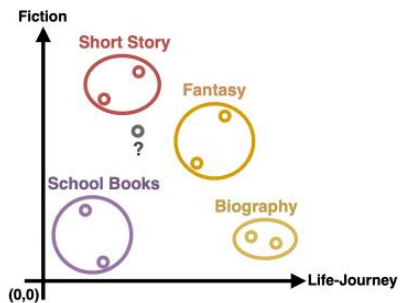


Embedding

Que ocurre cuando nuestros datos son textos.



216	16	16	16	24
224	24	224	224	232
232	32	232	32	40
240	40	240	240	48
248	48	48	48	56



Embedding

Mapear cada carácter con su número en la tabla ASCII

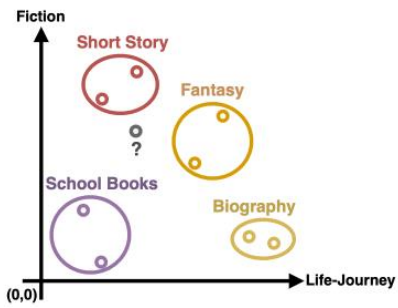


But here is the payment I left for you to find

66,117,116,32,104,101,114,101,32,105,115,32,
116,104,101,32,112,97,121,109,101,110,116,32,
73,32,108,101,102,116,32,102,111,114,32,
121,111,117,32,116,111,32,102,105,110,100

El objetivo es encontrar embeddings similares.

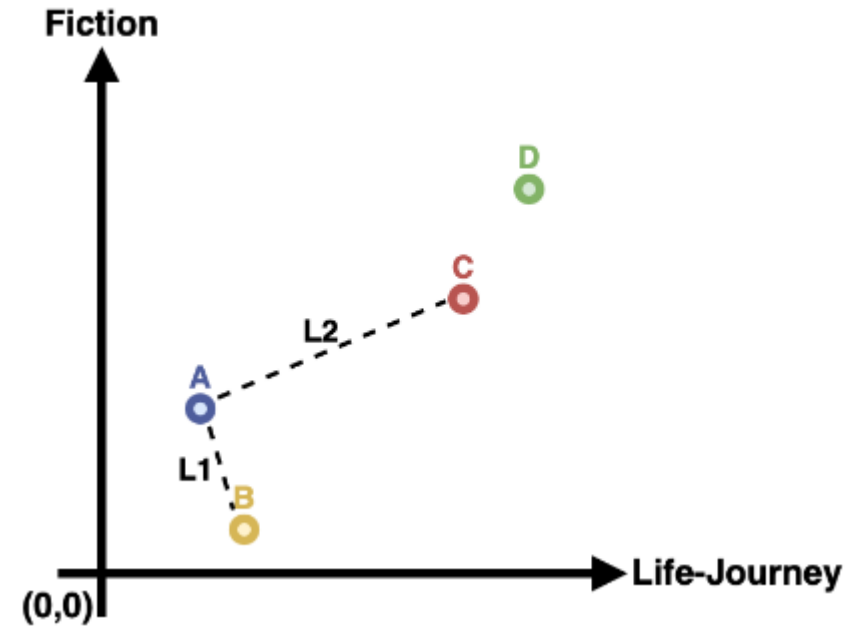
Number	Character	Number	Character	Number	Character	Number	Character	Number	Character	Number	Character
0	NULL	22	SYN	44	,	66	B	88	X	110	n
1	SOH	23	ETB	45	-	67	C	89	Y	111	o
2	STX	24	CAN	46	.	68	D	90	Z	112	p
3	ETX	25	EM	47	/	69	E	91	[	113	q
4	EOT	26	SUB	48	0	70	F	92	\	114	r
5	ENQ	27	ESC	49	1	71	G	93	]	115	s
6	ACK	28	FS	50	2	72	H	94	^	116	t
7	BEL	29	GS	51	3	73	I	95	_	117	u
8	BS	30	RS	52	4	74	J	96	`	118	v
9	TAB	31	US	53	5	75	K	97	a	119	w
10	LF	32	Space	54	6	76	L	98	b	120	x
11	VT	33	!	55	7	77	M	99	c	121	y
12	FF	34	"	56	8	78	N	100	d	122	x
13	CR	35	#	57	9	79	O	101	e	123	{
14	SOH	36	\$	58	:	80	P	102	f	124	
15	SI	37	%	59	;	81	Q	103	g	125	}
16	DLE	38	&	60	<	82	R	104	h	126	-
17	DC1	39	'	61	=	83	S	105	i	127	DEL
18	DC2	40	(	62	>	84	T	106	j		
19	DC3	41	)	63	?	85	U	107	k		
20	DC4	42	*	64	@	86	V	108	l		
21	NAK	43	+	65	A	87	W	109	m		

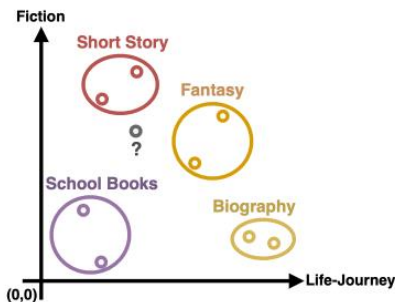


Embedding

Mapear cada carácter con su número en la tabla ASCII

Si los libros A y B son ficción la distancia entre ellos es más corta.



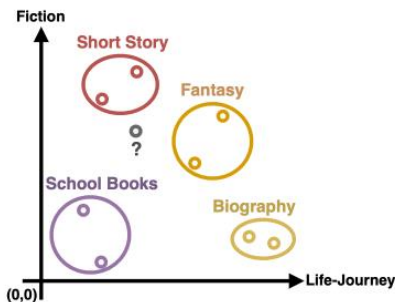


Embedding:TF-IDF

El algoritmo **TF-IDF** (Term Frequency-Inverse Document Frequency) es una técnica estadística para medir la importancia de una palabra en un documento dentro de una colección (corpus), calculando la frecuencia con la que aparece en el texto (TF) y penalizando su aparición en otros documentos (IDF), identificando así palabras únicas y relevantes, muy usado en recuperación de información.

TF-IDF vectorizer, que convierte texto (títulos/sinopsis/tags) en una matriz numérica para poder usar K-means.

```
vectorizer = TfidfVectorizer(  
    max_features=20000,  
    ngram_range=(1, 2),  
    stop_words="english",  
)
```



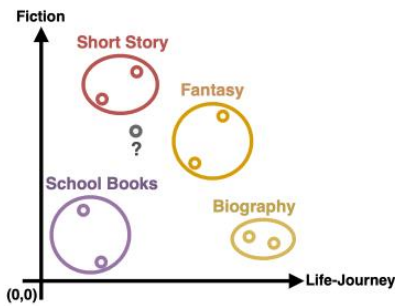
Embedding:TF-IDF

TF-IDF vectorizer, que convierte texto (títulos/sinopsis/tags) en una **matriz numérica** para poder usar K-means.

TfidfVectorizer(...) Hace 2 cosas a la vez:

1. tokeniza el texto (lo separa en términos)
1. calcula para cada documento un peso TF-IDF por término

Resultado: una matriz ($n_{\text{películas}} \times n_{\text{términos}}$), normalmente sparse.



Embedding:TF-IDF

Queremos convertir texto (títulos, sinopsis, tags) en números para poder usar ML (clustering, clasificación, búsqueda).

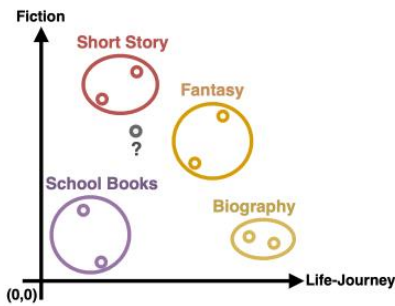
La idea: una palabra **es importante** en un documento si:

- aparece **mucho** en ese documento (**TF**) $: tf(t, d) = \#(t \text{ en } d)$
- pero aparece **poco** en el resto de documentos (**IDF**)

- N = nº de documentos
- $df(t)$ = en cuántos documentos aparece t

$$idf(t) = \log \left(\frac{1 + N}{1 + df(t)} \right) + 1$$

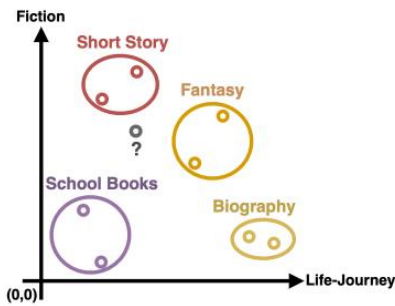
$$tfidf(t, d) = tf(t, d) \cdot idf(t)$$



Embedding:TF-IDF

- Palabras como **“the”, “and”, “movie”** aparecen en casi todo → **idf bajo** → aportan poco.
- Palabras como **“spaceship”, “wizard”, “courtroom”** aparecen en pocos docs → **idf alto** → ayudan a separar temas.

TF-IDF te da un vector donde cada dimensión es un término, y el valor indica **“qué tan característico** es ese término para ese documento”.

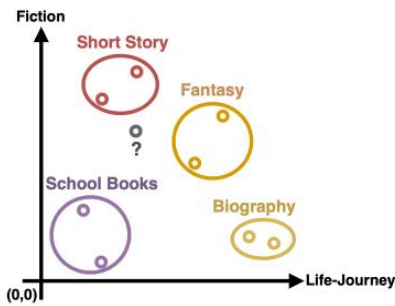


Embedding:TF-IDF

Si tienes 10.000 películas (en texto) y un vocabulario de 20.000 términos (features), obtienes:

- Matriz **XXX** de tamaño **(n_docs × n_terms)**
- Normalmente **sparse** (casi todo ceros).

Cada fila es el “embedding” TF-IDF de una película.



Embedding:TF-IDF

Ejemplo:

Documentos:

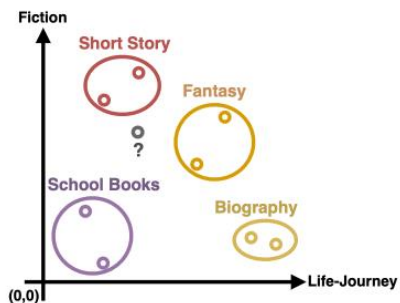
1. “space station alien”

2. “space station mission”

3. “romantic letters reunion”

- “space” y “station” aparecen en 2 documentos → idf medio
- “alien”, “mission”, “romantic”, “letters”, “reunion” aparecen en 1 → idf más alto

Así, doc1 queda más “marcado” por **alien**, doc2 por **mission**, doc3 por **romantic/letters/reunion**.



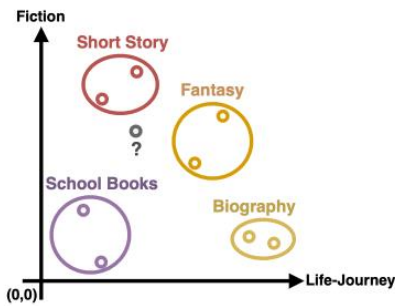
Embedding:TF-IDF

Matriz TF (conteos)

Doc	space	station	alien	mission	romantic	letters	reunion
D1	1	1	1	0	0	0	0
D2	1	1	0	1	0	0	0
D3	0	0	0	0	1	1	1

Matriz TF-IDF (TF * IDF)

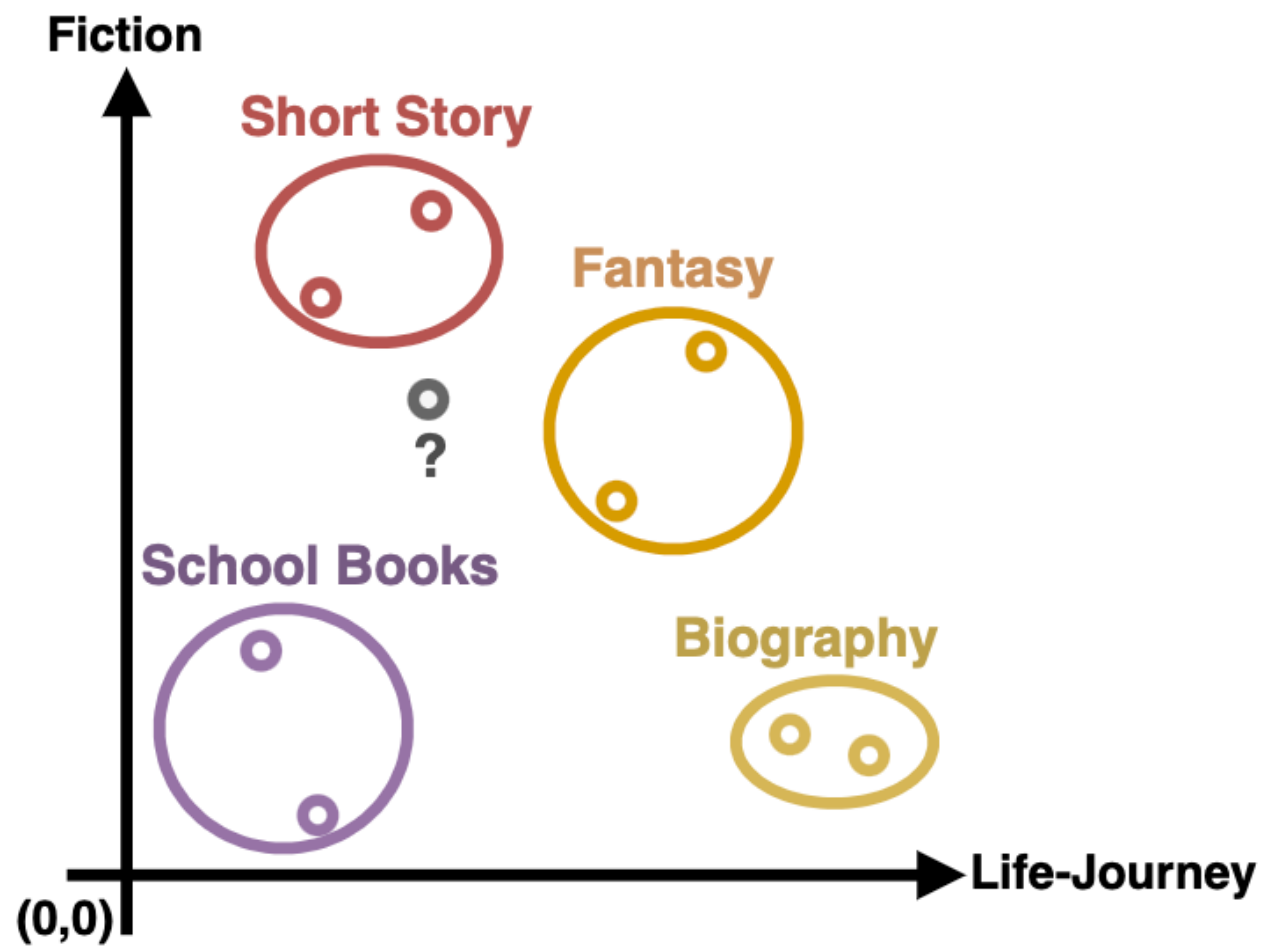
Doc	space	station	alien	mission	romantic	letters	reunion
D1	0.405	0.405	<u>1.099</u>	0.000	0.000	0.000	0.000
D2	0.405	0.405	0.000	<u>1.099</u>	0.000	0.000	0.000
D3	0.000	0.000	0.000	0.000	<u>1.099</u>	<u>1.099</u>	<u>1.099</u>



Embedding:TF-IDF

En resumen, TF-IDF es una técnica para crear embeddings (representaciones numéricas) de texto.

- Cada documento (una película) se convierte en un **vector** de tamaño = vocabulario (palabras/bigramas).
- Los valores del vector son los **pesos TF-IDF**.
- Normalmente ese vector es **sparse** (casi todo ceros), pero sigue siendo un embedding.



Gracias